

QCC Labs Cryptography

Author: @MicahKBrown (X)

Goal: Learn some OSINT fundamentals and about Tsurugi

Pre Requirements:

- Personal laptop with access to the internet

Cryptography is the science of securing communication and information using coded algorithms (ciphers) to make data unreadable to unauthorized parties, protecting confidentiality, integrity, and authenticity, and is fundamental to online security for everything from bank transactions to private messages, using techniques like symmetric (single key) and asymmetric (public/private key) encryption. It transforms plain text into scrambled ciphertext and back again using keys, ensuring privacy and preventing tampering.

Core Goals

- Confidentiality: Keeping information secret from eavesdroppers (privacy).
- Integrity: Ensuring data hasn't been altered or tampered with.
- Authentication: Verifying the identity of users or data sources.
- Non-repudiation: Preventing senders from denying they sent a message.

Main Types of Cryptography

- Symmetric-Key (Conventional): Uses the same single secret key for both encryption and decryption (e.g., AES).
- Asymmetric-Key (Public-Key): Uses a pair of keys: a public key for encryption and a private key for decryption (e.g., RSA, used in SSL/TLS for HTTPS).
- Hash Functions: Create a unique, fixed-size "fingerprint" of data to check for integrity (e.g., SHA-256).

CyberChef – The Swiss Army Knife

CyberChef, the "Cyber Swiss Army Knife," is a versatile, open-source web tool for cybersecurity professionals to easily encode, decode, encrypt, decrypt, compress, and manipulate data in a browser without complex scripting, making it essential for threat hunting, malware analysis, and forensics by simplifying tasks like Base64 decoding, hashing, and data extraction. Users chain operations into "recipes" for complex transformations, helping both technical and non-technical analysts understand and process digital data.

https://gchq.github.io/CyberChef/

Many of the labs will be leveraging CyberChef as our main tool!

Challenge 0: Base 64

Challenge Text: aHR0cHM6Ly9lbi53aWtpcGVkaWEub3JnL3dpa2kvSGlzdG9yeV9vZl9jcnlwdG9ncmFwaHk=

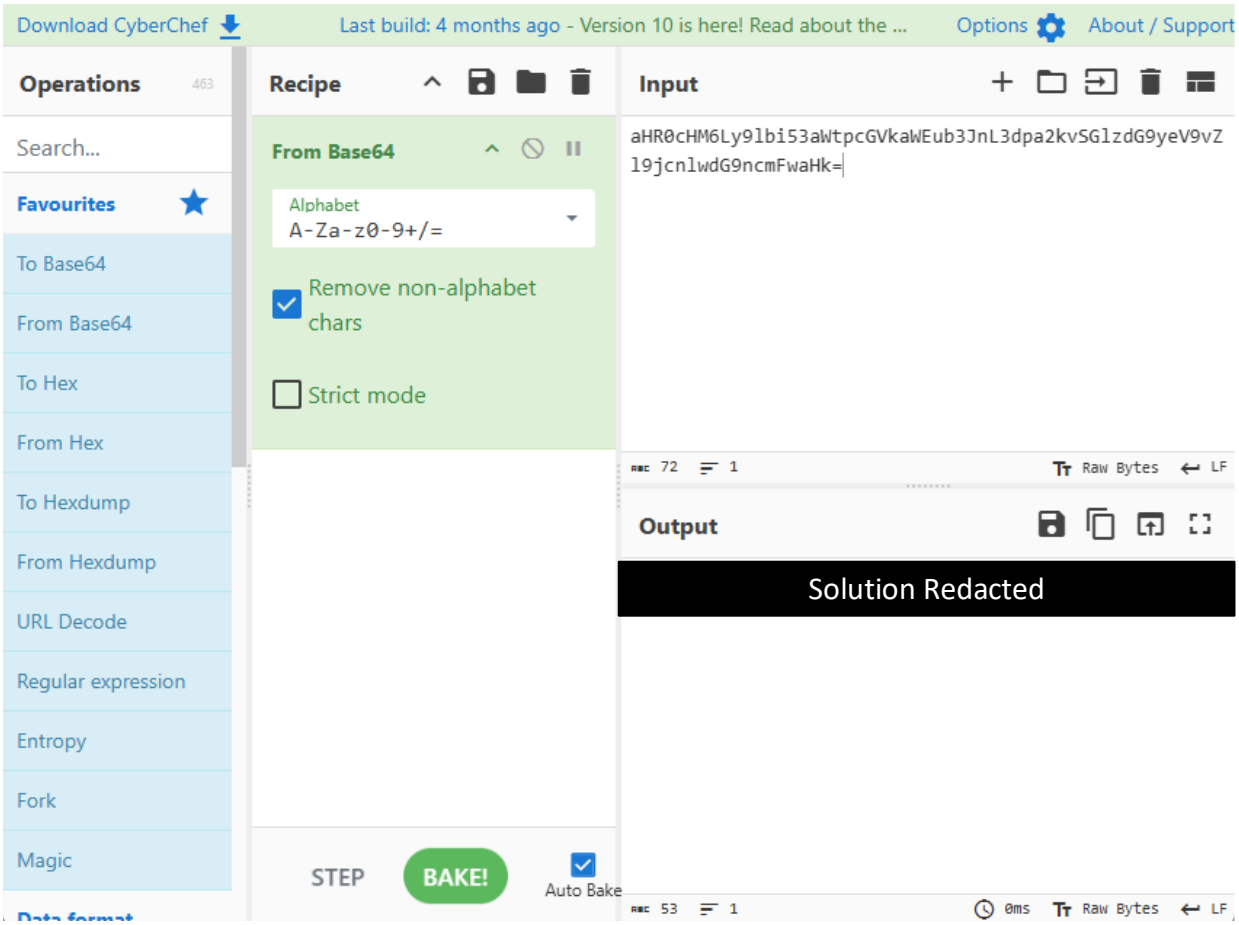
This is a very common challenge in many CTF. Note that this text ends with a equals sign. This is often a good indicator that the challenge text is in Base64

> “
> Base64 is a binary-to-text encoding that uses 64 printable characters to represent each 6-bit segment of a sequence of byte[1] values. As for all binary-to-text encodings, Base64 encoding enables transmitting binary data on a communication channel that only supports text.
>
> When comparing the original data to the resulting encoded data, Base64 encoding increases the size by 33% plus about 4% additional if inserting line breaks for typical line length.
>
> The earliest uses of this encoding were for dial-up communication between systems running the same operating system – for example, uuencode for UNIX and BinHex for the TRS-80 (later adapted for the Macintosh) – and could therefore make more assumptions about what characters were safe to use. For instance, uuencode uses uppercase letters, digits, and many punctuation characters, but no lowercase
>
> “
>
> https://en.wikipedia.org/wiki/Base64

While we can use Base64 to obfuscate text, it is easily reversable bust by using the ‘from Base64’. As such Base64 is often called encoding and not encryption.

To Solve this challenge:

1. Simply drag and drop ‘From Base 64’ into the Recipe.
2. Paste the challenge text into the ‘Input’ field.
3. Click ‘Bake’ if ‘Auto Bake is not selected in the bottom right of the Recipe area.



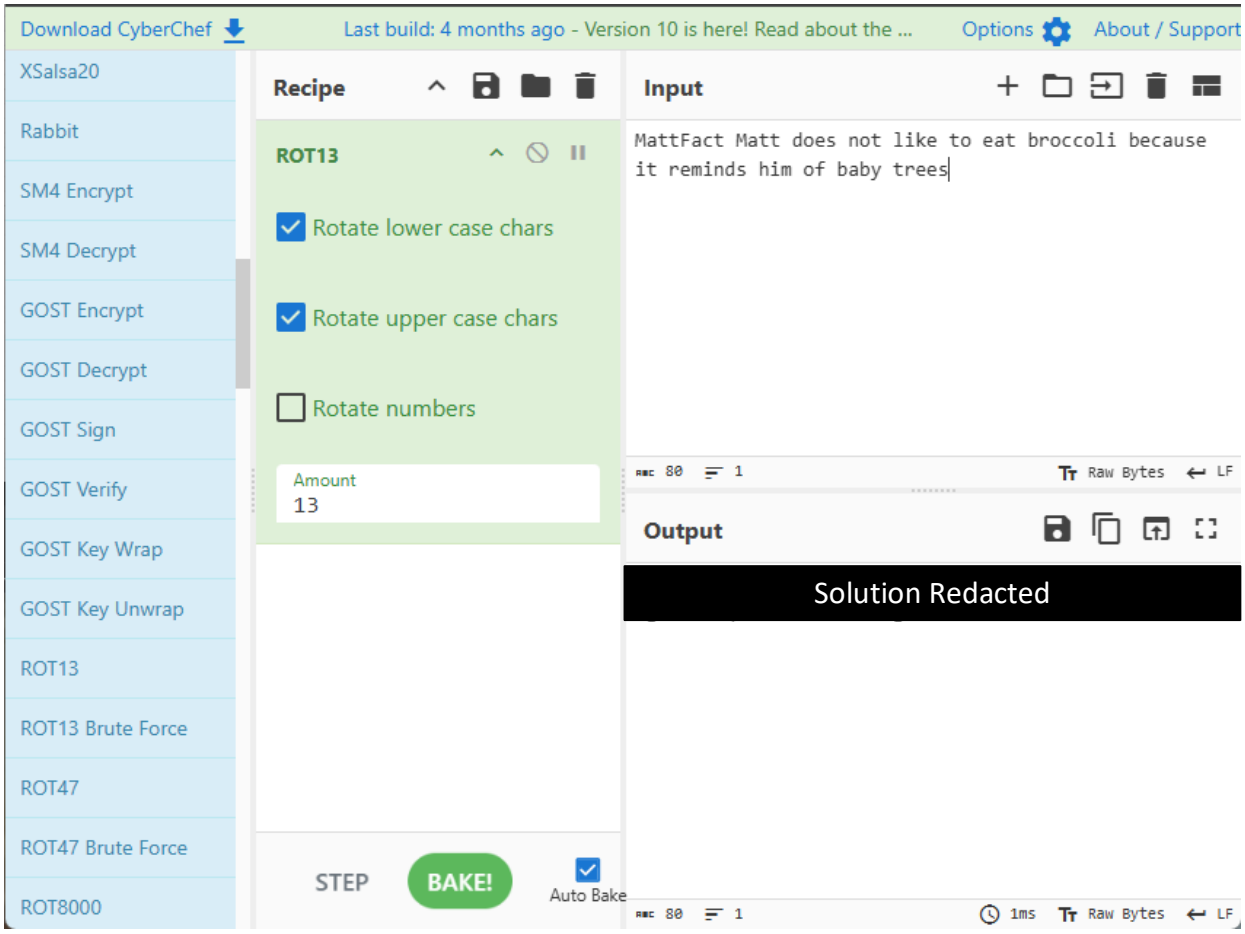
Challenge 1: ROT13

Challenge Text: ZnggSnpg Zngg qbrf abg yvxr gb rng oebppbyv orpnhfr vg erzvaqf uvz bs onol gerrf

ROT13 is as an online obfuscation technique, popularized on Usenet and forums to hide spoilers (movie endings, game answers), offensive content, or puzzle solutions, allowing users to decode it if they choose to, while fooling simple spam bots, rather than for actual security. Its simplicity and reversibility (applying it twice returns the original text) made it a staple for "mildly secret" messages in early internet culture, with some text editors even having built-in one-click ROT13 functions.

To Solve this challenge:

1. Simply drag and drop ‘ROT13’ into the Recipe.
2. Paste the challenge text into the ‘Input’ field.
3. Click ‘Bake’ if ‘Auto Bake is not selected in the bottom right of the Recipe area.



Challenge 2: Vigenère

Challenge Text: ttkxf://ip.kzzabeuqn.stu/nxcu/Cflr_xczbtj
Key: MarineCorps

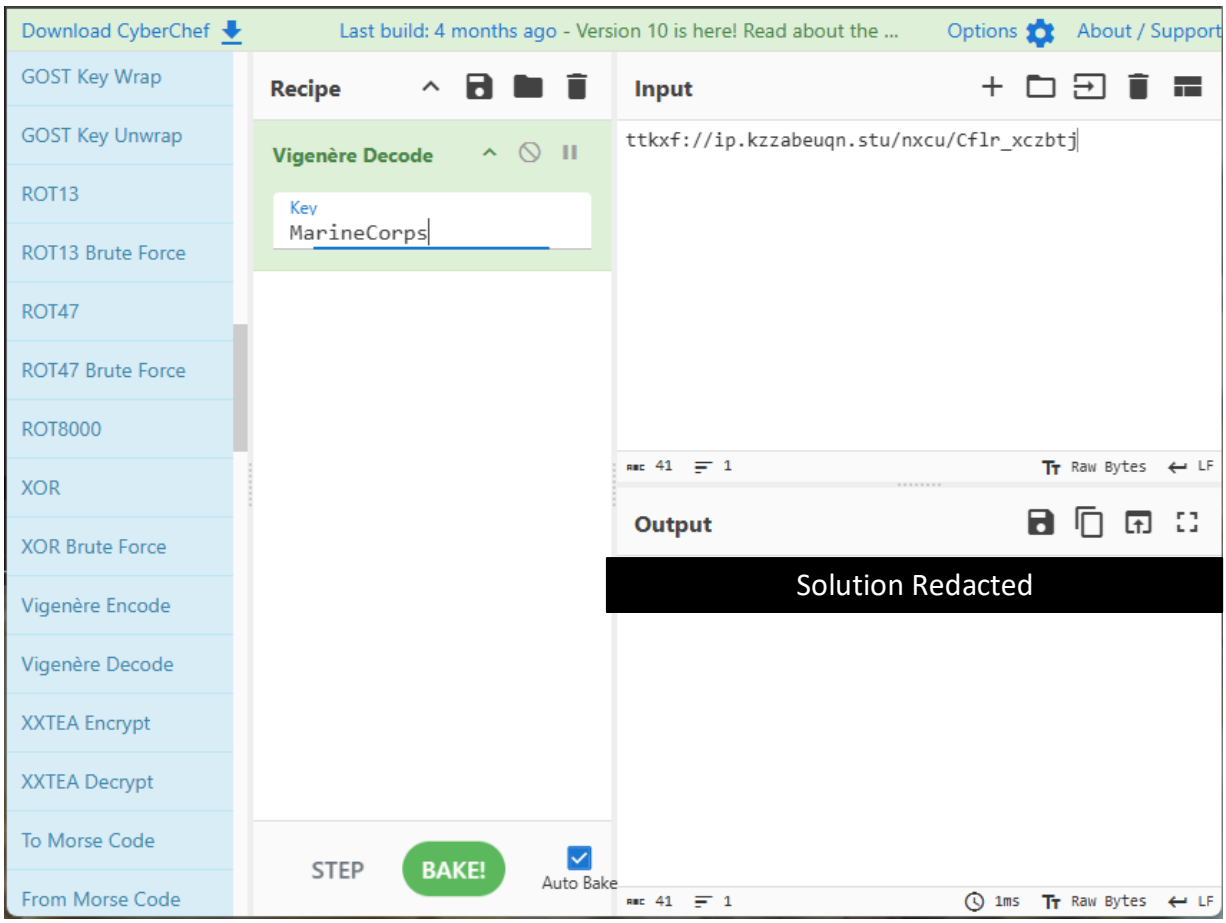
The Vigenère cipher is a method of encrypting alphabetic text using a series of interwoven

Caesar ciphers based on a keyword. While modern Artificial Intelligence (AI) can easily solve or implement this historic cipher, the "AI answer" for how it works involves a system of polyalphabetic substitution to enhance security over simpler ciphers like the Caesar cipher. For more information:

https://en.wikipedia.org/wiki/Vigen%C3%A8re_cipher

To Solve this challenge:

1. Simply drag and drop 'Vigenère Decode' into the Recipe.
2. Paste the challenge text into the 'Input' field.
3. Click 'Bake' if 'Auto Bake is not selected in the bottom right of the Recipe area.



Challenge 3:Morse Code

Challenge Text:

-. . ...- . .-.

- .-. ..- ... -

-- .. -.-. .- .... .----. ...

- .- ... - .

.. -.

-- ..- ... .. -.-. -.-.--

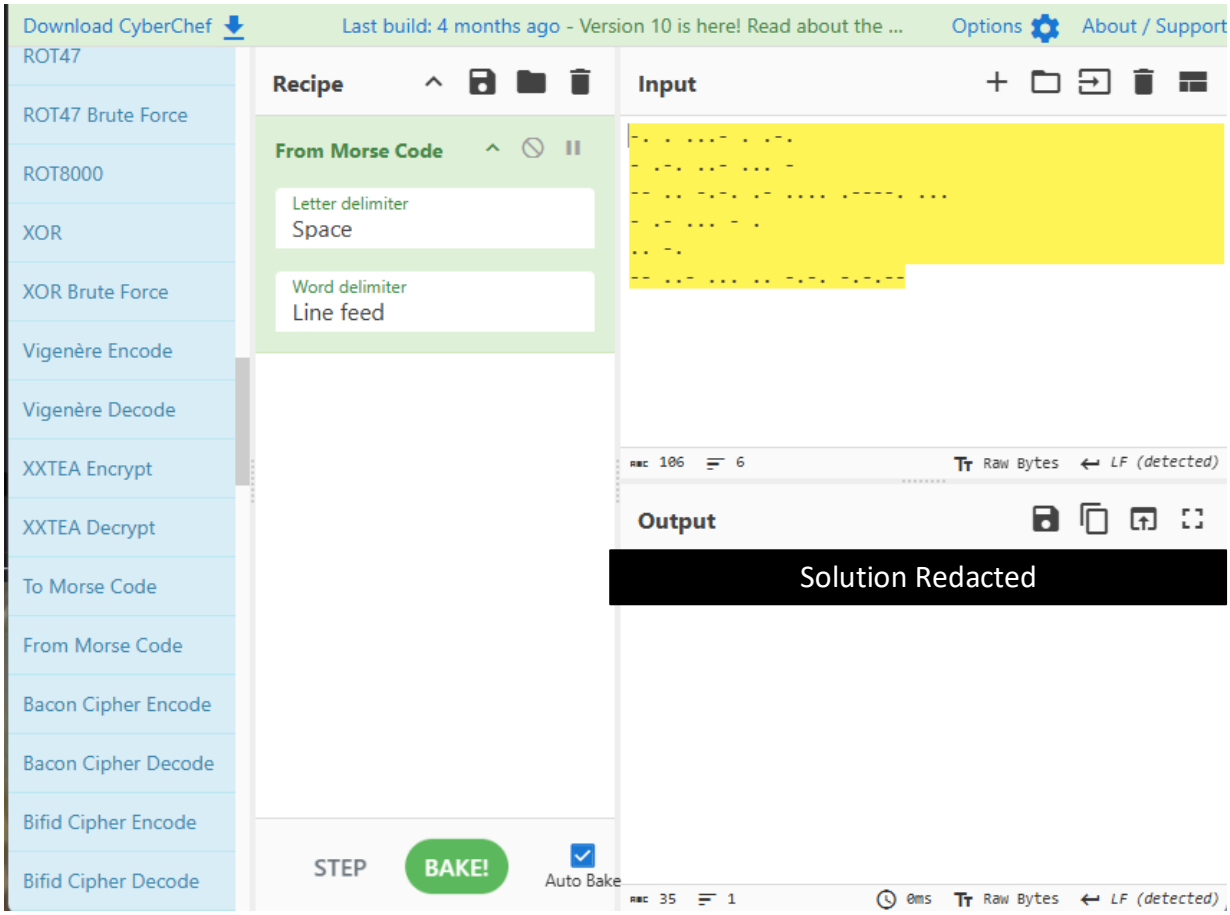
Morse code is a communication system using standardized short (dots/dits) and long (dashes/dahs) signals to represent letters, numbers, and punctuation, historically transmitted via electrical pulses (telegraphy) but also as sounds or lights, with the International Morse Code being the global standard since the 1850s for long-distance messaging, distress signals (SOS), and even aircraft identification. For more information

https://en.wikipedia.org/wiki/Morse_code

To Solve this challenge:

1. Simply drag and drop 'From MorseCode' into the Recipe.

2. Paste the challenge text into the ‘Input’ field.
3. Click ‘Bake’ if ‘Auto Bake is not selected in the bottom right of the Recipe area.



Challenge 4: Logical Operators

Equals / ==

- Two equals signs placed together ask if two values are equal to each other. For example, the statement (A == B) is true when a is equal to b.

NOT Equalls / !=

- An exclamation point and an equals sign placed together determine if two values are not equal to each other. That is, is the value or expression to the left of the! = sign not equal to the value or expression to the right? For example, the statement (A != b) is true if A does not equal B.

Less Than / <

- A < sign asks if the first value is less than the second value. That is, is the value or expression to the left of the < sign less than the value or expression to the right side? For example, the statement (A < B) is true if A is less than B.

Less Than or Equals / <=

- A < than sign placed together with an equals sign asks if the first value is less than or equal to the second value. That is, is the value or expression to the left of the <= sign less than or equal to the value or expression to the right side? For example, the statement (A <= B) is true if A is less than or equal to B.

Greater Than / >

- A > sign asks if the first value is greater than the second value. That is, is the value or expression to the left of the > sign greater than the value or expression to the right side? For example, the statement (A > B) is true if A is greater than B.

Greater than or Equals / >=

- A > sign followed by an equals sign asks if the first value is greater than or equal to the second value. That is, is the value or expression to the left of the >= signs greater than or equal to the value or expression to the right side? For example, the statement (A >= B) is true if A is greater than or equal to B.

XOR (Exclusive OR) / (A && !B) || (!A && B)
XOR is a logical/bitwise operation that outputs true (or 1) if exactly one of its inputs is true (or 1), and false (or 0) otherwise; it's like "either/or but not both," used in computing for tasks like simple encryption (XORing data with a key reverses the process), error detection, and efficient data manipulation, acting as a binary addition without carries.
How it works (Binary)

- 0 XOR 0 = 0 (Both false, so false)
- 0 XOR 1 = 1 (One true, so true)
- 1 XOR 0 = 1 (One true, so true)
- 1 XOR 1 = 0 (Both true, so false)

Now use Cyberchef to crack this text.
CipherText: 9w%s"9~, v4f?`8w(`>m⍰l#d~
Key: 513

Challenge 5: Hashing

Hashing is converting data (like a password or file) into a unique, fixed-length string of characters (a "hash"). Using a hash function, primarily for verifying data integrity and securely storing passwords without revealing the original data. This is due to the hash is a ‘one way’ repeatable process in that every time you hash the file / data you will get a seporate unique output (hash) that is unique to that file / data and proves that it has not been changed since the last time it was hashed.

*While it is possible for two different files / pieces of data to produce the same hash, this is extremely rare and often has to be manipulated.

While Hashing is closely related to encryption the key differentiator in my mind is that encryption is a ‘two way process’ where the output of an encryption function can be processed again and transformed back into the original state. Hashing is a ‘one way process’ where (under strong hashing protocols) you can not use a hash and hashing function to recreate the original document.

Hashing Concepts

- Hash Function: A mathematical algorithm that processes input data (any size) to produce a fixed-size output.
- Hash Value (Digest): The unique, fixed-length output from the hash function.
- One-Way Process: Hashing is irreversible; you can't get the original data back from the hash, making it ideal for security.

Key Uses in Cybersecurity

- Password Storage: Instead of storing passwords, systems store their hashes. When a user logs in, their entered password is hashed and compared to the stored hash.
- Data Integrity: Verifying a downloaded file hasn't been altered by comparing its hash to a known good hash (e.g., SHA-256 checksums).
- Digital Signatures: Hashing ensures a message's authenticity and integrity by encrypting the hash, not the entire message, with a private key.
- Blockchain: Secures transactions and links blocks in a chain, ensuring immutability.

Hashing vs. Encryption

- Hashing: One-way, irreversible, used for verification (passwords, integrity).
- Encryption: Two-way, reversible with a key, used for confidentiality (protecting data in transit or storage).

Some Common Hashes and their uses:

- SHA-2 Family (SHA-256, SHA-512): Very common, strong, used in TLS/SSL, digital signatures, and cryptocurrencies.
- SHA-3 (Keccak): Newer standard, built for future needs, including post-quantum security.
- bcrypt & scrypt: Specialized for passwords; memory-intensive to resist brute-force attacks.
- MD5: Older, now considered insecure for many uses due to vulnerabilities, but historically used for integrity checks.

Challenge:

1. In Cyberchef, search for and add MD5 to the Recipe
2. In the input add the following text:

We're no strangers to love
You know the rules and so do I
A full commitment's what I'm thinking of
You wouldn't get this from any other guy

I just wanna tell you how I'm feeling
Gotta make you understand

Never gonna give you up
Never gonna let you down
Never gonna run around and desert you
Never gonna make you cry
Never gonna say goodbye
Never gonna tell a lie and hurt you

3. Click Bake if ‘auto bake’ is not turned on.
4. Note the output in a text file.
5. Change a single character in the input. Compare the old output to the new. You will see they have changed

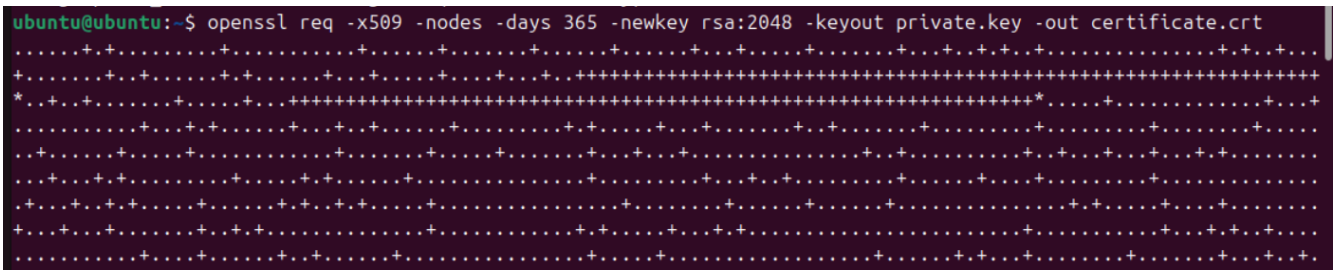
Challenge 6: TLS (optional)

For this you will need to have a Linux with OpenSSL installed. I suggest using a default Ubuntu as it has OpenSSL installed by default.

An RSA 2048 certificate uses a 2048-bit key for strong encryption, balancing robust security with performance, and is a widely supported standard for SSL/TLS to secure websites and data, generated with tools like OpenSSL by creating a public/private key pair, though NIST recommends transitioning to larger keys (like 3072-bit or 4096-bit) as quantum computing threats emerge, by {2030-2035}.

Here we are going to use the basic command.

openssl req -x509 -nodes -days 365 -newkey rsa:2048 -keyout private.key certificate.crt



Here you can see I am being asked for some more information on the certificate. Then you can see with the ls command I have certificate.crt and private.key.



I can use the cat command to view each of these files:

```
ubuntu@ubuntu:~$ cat certificate.crt
-----BEGIN CERTIFICATE-----
MIIDnzCCAoegAwIBAgIUaDqsnxD53Q9j5nY1+oZy5VGgyqwwDQYJKoZIhvcNAQEL
BQAwXzELMAkGA1UEBhMCVVMxDTALBgNVBAgMBE9oaW8xEzARBgNVBAcMCkNpbmNp
bm5hdGkxFTATBgNVBAoMDFF1ZWVuQ2l0eUNvbjEVMBMGA1UEAwwMVGhlV2lyZVNo
YXJrMB4XDTI1MTIxMDE4MTIxOVoXDTI2MTIxMDE4MTIxOVowXzELMAkGA1UEBhMC
VVMxDTALBgNVBAgMBE9oaW8xEzARBgNVBAcMCkNpbmNpbm5hdGkxFTATBgNVBAoM
DFF1ZWVuQ2l0eUNvbjEVMBMGA1UEAwwMVGhlV2lyZVNoYXJrMIIBIjANBgkqhkiG
9w0BAQEFAAOCAQ8AMIIBCgKCAQEArh3GLW0eDnYlqOhZBdhoRxlmCRnIjCgC45md
R23UCdBxOU0/iM1dpCWdAC4kPcMjAlbOIfNtuektRR706LMyAGo7T1U5JoAKSDaD
292YGUtaEYF/7mCc5w9rg5ZzCUqbQjWwwP6WiaoGc5lavZ5b/qHjAx6OEEqMhL/3
+NLZUY2ynORhsXygsJhPQxkH8yQjsA0btP4wP/w2G4MmLDosQWqUIdKy3VfVZmpA
kHJQyFAghhXP4gzF37qJ2vJKHiMqfehTI/xOsUHbAotzkH8WjGksF8sS5Y/qYoBT
Eh5XPNxJjFpugRE5oqKtde2huPRtFQmfVA+fiB32W+amPQGnGQIDAQABo1MwUTAd
BgNVHQ4EFgQUzwYBjlJelkWsh1INeC/MXNsU+LEwHwYDVR0jBBgwFoAUzwYBjlJe
lkWsh1INeC/MXNsU+LEwDwYDVR0TAQH/BAUwAwEB/zANBgkqhkiG9w0BAQsFAAOC
AQEAAKl95lrF3gvPlutlDRLjw4ozr+jaz5ZAgpW1OwwFXPJcgUw4XRkZwgraHP69
c2mN3XiPVB1XbOnjVzWQ6s5A44Qq4V0XhMQx3TZTirGhWmiN9wGxsXrFYuDOtkjH
InrtrSjP05SH314KA3gmeZtEL3Jh7M0ToNSb6HUSR3mIG76564Hg7LNbk1WeSsaO
31rPeAFRIyndwJQopyjf9BeDwvExV26t5ZnNHDmEJLCpcjNR/MGT+NkH/iBk+xYx
dUkKBe8mHjEOtpsiYZGpfu7Qh7Bsahtz9YOOvSR+6Q1toxoeWRZ1JnuI7xlmVZKA
pTFzAV6TJN/Sa1FdMXfRHz/wwA==
-----END CERTIFICATE-----
ubuntu@ubuntu:~$
```

```
ubuntu@ubuntu:~$ cat private.key
-----BEGIN PRIVATE KEY-----
MIIEvgIBADANBgkqhkiG9w0BAQEFAASCBKgwggSkAgEAAoIBAQCuHcYtbR4OdiWo
6FkF2GhHGWYJGciMKALjmZ1HbdQJ0HE5TT+IzV2kJZ0ALiQ9wyMCVs4h82256S1F
HvToszIAajtPVTkmgApINoPb3ZgZS1oRgX/uYJznD2uDlnMJSptCNbDA/paJqgZz
mVq9nlv+oeMDHo4QSoyEv/f40tlRjbKc5GGxfKCwmE9DGQfzJCOwDRu0/jA//DYb
gyYsOixBapQh0rLdV9VmakCQclDIUCCGFc/iDMXfuona8koeIyp96FMj/E6xQdsC
i3OQfxaMaSwXyxLlj+pigFMSHlc83EmMWm6BETmioq117aG49G0VCZ9UD5+IHfZb
5qY9AacZAgMBAAECggEAIoGMENybJEqN1BeX7OyiXdTOv+b4M66KRAjhfpJpfzhr
9+szOC1a7Xj20F5UmFjfy0rgtyHDESEA873a03MP6b0j2+kL1nFxmDYVk7Eqv3iL
dQSxEdDbx37q070xNxouAsUZLmZF9ypW9PnjXcjQenoivy+C//+GMZjJz+oZX2vd
11cHdnFbW138e2gn2qhQPzwiak7ls5Um3mbep5Tbr8sUbABbR4Rrl7UYXshBau63
Iz0cME1yxGWHIrmhS2dJCHALUpemqcr7cJE80rww3FmY5onZMr20HjLXdaofAPYz
63GFtI+AzjsPLA5/OToQ+FnF0ytBiKUpq0LONiOSMQKBgQDTXHx6fjMXO34TRd0i
cqMmDtfXptIL597AGf/Uruix3w8wyHeIGHocAlPlI5cnsz1FVj1lbi0ZsL2FvW8O
0XnPdHwdCQxBYrc6RpEIQGDjrebugt8BU8FGnku/mWwLRxHS3vvMCs93FxeqkQIu
cPMSR5qY/MsP6TJmLmHTKr7J3wKBgQDS45d37N186EOhHbRNhJmwnCeX+VUPj1O8
GvHVgGcYXumJNhkww3PvjYtvnHbel7ZVbq11ThN9eQPicdvFVNmE/HisUPb3H61L
R0dXcgACm3HRF6q2u2DqBD7fJlqwqNE6A+jmnoV1UxhEvqYBM8vbRG5Z5RVSID8r
```

Rember you can see all the OpenSSL flags with this command:

Openssl –help

```
ubuntu@ubuntu:~$ openssl --help
help:

Standard commands
asn1parse         ca                ciphers           cmp
cms               crl               crl2pkcs7         dgst
dhparam           dsa               dsaparam          ec
ecparam           enc               engine            errstr
fipsinstall       gendsa            genpkey           genrsa
help              info              kdf               list
mac               nseq              ocsp              passwd
pkcs12            pkcs7             pkcs8             pkey
pkeyparam         pkeyutl           prime             rand
rehash            req               rsa               rsautl
s_client          s_server          s_time            sess_id
smime             speed             spkac             srp
storeutl          ts                verify            version
x509

Message Digest commands (see the `dgst' command for more details)
blake2b512        blake2s256        md4               md5
rmd160            sha1              sha224            sha256
```

As this keypair was not signed by a Certificate Authority. You will have to install / trust this cert if it were to be used.

Challenge 7: TLS Handshake

Ok for the next part we need to partner up! One of you will assume the role of the server and the other the client.

Each person needs three objects to represent their TLS Public Key and TLS Private Key.

I know this might be a tad silly and overly dramatic, but hey you have put up with us this long!

1. ClientHello: The client sends its supported TLS versions, cipher suites (encryption methods), and a random string (Client Random).
    a. Client Instruction: At this point the ‘Client’ will hand over the a random (clean) word to represent the ‘Server’ to represent the ‘Client Random’
2. ServerHello: The server picks the best TLS version and cipher suite, sends its own random string (Server Random), its digital certificate (containing its public key), and a "ServerHelloDone" message.
    a. Client Instruction: At this point the ‘Server’ will hand over a random (clean) word to represent the ‘Server Random’ and the object specified as the ‘Server Public Key’ to the ‘Client’
3. Certificate Verification: The client checks the server's certificate against its list of trusted Certificate Authorities (CAs) to confirm the server's identity.
    a. Client Instruction: If the certificate public key is not either signed by a trusted CA or has been installed as a trusted certificate (i.e. self-signed) the client will be asked to trust the certificate or be denied access based on local config. (for the purpose of tis excersise we wither trust the certs signer or trust the cert at this step.)
4. Client Key Exchange: The client generates a "pre-master secret," encrypts it using the server's public key (from the certificate), and sends it.
    a. Client Instruction: The ‘Client’ will now hand over another random (clean) word that represents the ‘pre-master secret’ encrypted by servers public key (which can only be decrypted by the Server’s private key.)

5. Key Derivation: The server decrypts the pre-master secret using its private key. Both parties then use the Client Random, Server Random, and pre-master secret to independently derive the same master secret, from which symmetric session keys are generated.
    a. Client Instruction: ‘Server’ decrypts ‘pre-master key’ with Server Private Key. Both Server and Client clap hands to represent the dank encryption magics.
6. Change Cipher Spec (Client): The client sends a message indicating it's switching to encrypted communication.
    a. Client Instruction: ‘Client’ notifies ‘Server’ it wants to switch to encrypted communications.
7. Finished (Client): The client sends an encrypted "Finished" message, a hash of all previous handshake messages, to confirm everything is in order.
    a. Client Instruction: ‘Client’ sends ‘Server’ “Finished” messaged
8. Change Cipher Spec & Finished (Server): The server does the same – sends its "Change Cipher Spec" and an encrypted "Finished" message, verifying the handshake.
    a. Client Instruction: ‘Server’ sets Ciper Sec and sends ‘Finished’ message to ‘Client’

All objects exchanged are returned to their owners.

Challenge 8: VeraCrypt

“

VeraCrypt is a free open source disk encryption software for Windows, Mac OSX and Linux. Brought to you by AM Crypto (https://amcrypto.jp) and based on TrueCrypt 7.1a.

VeraCrypt main features:

- Creates a virtual encrypted disk within a file and mounts it as a real disk.
- Encrypts an entire partition or storage device such as USB flash drive or hard drive.
- Encrypts a partition or drive where Windows is installed (pre-boot authentication).
- Encryption is automatic, real-time(on-the-fly) and transparent.
- Parallelization and pipelining allow data to be read and written as fast as if the drive was not encrypted.
- Encryption can be hardware-accelerated on modern processors.
- Provides plausible deniability, in case an adversary forces you to reveal the password: Hidden volume (steganography) and hidden operating system.
- More information about the features of VeraCrypt may be found in the documentation

“

https://veracrypt.io/en/Home.html

Download a version of VeraCrypt from the site. I downloaded the Windows portable app version and launch it.

https://veracrypt.io/en/Downloads.html

1. Select your language, accept the EULA, select location and click extract.
2. Launch Veracypt
3. You should now see the VeraCrypt app, sorry but as it is written I can not launch it. Select an unused drive and click ‘Create New Volume’.

4. Select ‘Create an encrypted file container’ and click next.
5. On the next screen I selected ‘Standard VeraCrypt volume’ and click next.
6. On the next screen tell VeraCrypt where to store the file and give it a name. then click next.
7. On ‘Encryption Options’ select that encryption and hash algorithms and click next. (AES / SHA-512)
8. On ‘Volume Size’ enter you encrypted file size. (I gave 50 MB as this is a demo)
9. On ‘Volume Password’ type and confirm a password for the encrypted file and click next.
10. On ‘Volume Format’ move your mouse to help create entropy (randomness).
11. Click Format when you have embodied enough randomness.
12. On ‘Volume Created’ click next
13. Cancel out of the wizard.
14. At the main widow click ‘Select File’ and point it to the encrypted file you just created.
15. Click an unused drive and click ‘Mount’. Enter the password you just used and click ‘OK’.
16. You should now see this drive in your file manager app (different per OS). In the VeraCrypt console you should now see your encrypted file mounted as a drive. Anything you store in that drive is automatically protected. (this is a great way to add robust encryption to portable media)
17. You can mount and unmount this file as needed.

Challenge 9: Kryptos

You know what to do but you have 5 minutes until we ‘shake it off’ and get back to the final lab challenge!

One of the most interesting cryptography puzzles in the work might just be sitting outside of CIA headquarters known as Kryptos. It was dedicated in 1990. The creator of this sculpture said that the ‘fifth code’ will reveal itself after the first four are solved. To this day only three of the five codes have been solved.



https://en.wikipedia.org/wiki/Kryptos

Left side (as seen from the courtyard)	Right Side (as seen from the courtyard)

EMUFPHZLRFAXYUSDJKZLDKRNSHGNFIVJ	ABCDEFGHIJKLMNOPQRSTUVWXYZABCD
YQTQUXQBQVYUVLLTREVJYQTMKYRDMFD	AKRYPTOSABCDEFGHIJLMNQUVWXZKRYP
VFPJUDEEHZWETZYVGWHKKQETGFQJNCE	BRYPTOSABCDEFGHIJLMNQUVWXZKRYPT
GGWHKK?DQMCPFQZDQMMIAGPFXHQRLG	CYPTOSABCDEFGHIJLMNQUVWXZKRYPTO
TIMVMZJANQLVKQEDAGDVFRPJUNGEUNA	DPTOSABCDEFGHIJLMNQUVWXZKRYPTOS
QZGZLECGYUXUEENJTBJLBQCRTBJDFHRR	ETOSABCDEFGHIJLMNQUVWXZKRYPTOSA
YIZETKZEMVDUFKSJHKFWHKUWQLSZFTI	FOSABCDEFGHIJLMNQUVWXZKRYPTOSAB
HHDDDUVH?DWKBFUFPWNTDFIYCUQZERE	GSABCDEFGHIJLMNQUVWXZKRYPTOSABC
EVLDKFEZMOQQJLTTUGSYQPFEUNLAVIDX	HABCDEFGHIJLMNQUVWXZKRYPTOSABCD
FLGGTEZ?FKZBSFDQVGOGIPUFXHHDRKF	IBCDEFGHIJLMNQUVWXZKRYPTOSABCDE
FHQNTGPUAECNUVPDJMQCLQUMUNEDFQ	JCDEFGHIJLMNQUVWXZKRYPTOSABCDEF
ELZZVRRGKFFVOEEXBDMVPNFQXEZLGRE	KDEFGHIJLMNQUVWXZKRYPTOSABCDEFG
DNQFMPNZGLFLPMRJQYALMGNUVPDXVKP	LEFGHIJLMNQUVWXZKRYPTOSABCDEFGH
DQUMEBEDMHDAFMJGZNUPLGEWJLLAETG	MFGHIJLMNQUVWXZKRYPTOSABCDEFGHI
ENDYAHROHNLSRHEOCPTEOIBIDYSHNAIA	NGHIJLMNQUVWXZKRYPTOSABCDEFGHIJL
CHTNREYULDSLLSLLNOHSNOSMRWXMNE	OHIJLMNQUVWXZKRYPTOSABCDEFGHIJL
TPRNGATIHNRARPESLNNELEBLPIIACAE	PIJLMNQUVWXZKRYPTOSABCDEFGHIJLM
WMTWNDITEENRAHCTENEUDRETNHAEOE	QJLMNQUVWXZKRYPTOSABCDEFGHIJLMN
TFOLSEDTIWENHAEIOYTEYQHEENCTAYCR	RLMNQUVWXZKRYPTOSABCDEFGHIJLMNQ
EIFTBRSPAMHHEWENATAMATEGYEERLB	SMNQUVWXZKRYPTOSABCDEFGHIJLMNQU
TEEFOASFIOTUETUAEOTOARMAEERTNRTI	TNQUVWXZKRYPTOSABCDEFGHIJLMNQUV
BSEDDNIAAHTTMSTEWPIEROAGRIEWFEB	UQUVWXZKRYPTOSABCDEFGHIJLMNQUVW
AECTDDHILCEIHSITEGOEAOSDDRYDLORIT	VUVWXZKRYPTOSABCDEFGHIJLMNQUVWX
RKLMLEHAGTDHARDPNEOHMGFMFEUHE	WVWXZKRYPTOSABCDEFGHIJLMNQUVWXZ
ECDMRIPFEIMEHNLSSTTRTVDOHW?OBKR	XWXZKRYPTOSABCDEFGHIJLMNQUVWXZK
UOXOGHULBSOLIFBBWFLRVQQPRNGKSSO	YXZKRYPTOSABCDEFGHIJLMNQUVWXZKR
TWTQSJQSSEKZZWATJKLUDIAWINFBNYP	ZZKRYPTOSABCDEFGHIJLMNQUVWXZKRY
VTTMZFPKWGDKZXTJCDIGKUHUAUEKCAR	ABCDEFGHIJKLMNOPQRSTUVWXYZABCD

The solution for the first (K1) and second (K2) passage was a Vigenère cypher as seen in challenge 2.

Passage three (K3) was solved by using a transposition cipher.

A transposition cipher rearranges the letters (or bits) of a message instead of changing them, hiding meaning by scrambling the order based on a secret key or pattern, like writing in a grid and reading columns out differently (columnar transposition) or in a zigzag (rail fence). While simple transpositions are weak alone, they are key components in modern, strong algorithms like AES and DES, often combined with substitution for robust encryption.

Passage four (K4) and the hinted at passage 5 (K5) have yet to be solved even though the creator (Jim Sanborn) has offered many hints over the year.

In September 2025, Sanborn announced in the Wahington Post plans to auction off the solution to the fourth passage. However things often get messy when large amounts of large amounts of money get involved. Sanborn donated documents to the Smithsonian which were able to be used to re-assemble K4 as confirmed by Sanborn. However this was not a true solve of K4 but a hack, and I have to respect the alternative solution method. Unfortunately the two people who reassembled K4 based off the notes in the Smithsonian were threatened with legal action by the auction house over

copyright infringement and interference with contracts if the text of K4 were to published. Given this, the two re-assemblers chose to sign an NDA that keeps the secret of K4.

Sanborn confirmed the authenticity of the reassembled message of K4 and has publicly stated that he did not care what method was used to 'solve' it.

For know the message of K4 and K5 are still waiting to be publicly solved. Feel free to use what you have learned here to attempt to crack one of the most well known cryptographic puzzles in the world. For more information on Kryptos and a good write up on the solves for K1, K2, and K3 please see this site:

https://en.wikipedia.org/wiki/Kryptos

Challenge 10: Derbycon

Micah here, one final challenge! No worries, everything you need to know to solve this challenge you have ALREADY done!

As you may know Challenge Coins are large in Government, Military (yes technically government, but with big boomsticks), and Hackers. In the hacker space they are often given out for an impressive feat, speaking / volunteering / training, or out of friendship. I was able to get this gem at DerbyCon IX and it is one of my most valued hacker treasures.

For the record, here is what it looks like and as you guessed they embedded a challenge in it.



You will see that each side has ciphertext on each side.

In truth the hardest part of this challenge was getting the string off the back of the coin and I even happened to have my jewelers loop in my truck!

Ciphertext dark side: 00110010000110000001100010011001O

Ciphertext light side:
CwwDBgERST8IGQQ/BggBHAREJQcJCigdAR0bDVQkAAACNRABHB8HVilFGg0jHQYAHQ9TNQYbBDQMQh0bDVQkAAACNRAB
HB0NVikbGQQoHRoGHwdTKUcqADQLFyocBkYjGwsTIxtA

So my first thought was the front and back have to work together. We can use the ‘From Binary’ on the Ciphertext Front to get the value 2012.

So this is where I started chase squirrels . . .

I had idea that I would need to do something with the two strings. In a way I was correct, but I was incorrect in one of the strings! After a bit of encouragement from some of my friends associated with the conference clued me in ‘not to overthink it’ and that everything I needed was on one side of the challenge coin.

It was at this time I knew to look on the light side and I knew I had pulled the text off character by character correctly. (Zoom and Enhance, Zoom and Enhance, Zoom and Enhance).

So now I tried to XOR the Ciphertext light side with several different phrases on the coin. . . EPIC FAIL AFTER EPIC FAIL. Using phrases like:

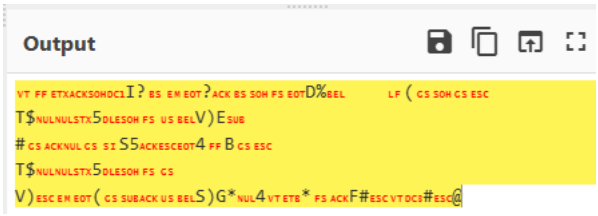
- DerbyCon IX
- DerbyCon IX Finish Line
- FinishLine
- BlackHat
- DerbyHat

  XOR (Exclusive OR) is a logical/bitwise operation that outputs true (or 1) if exactly one of its inputs is true (or 1), and false (or 0) otherwise; it's like "either/or but not both," used in computing for tasks like simple encryption (XORing data with a key reverses the process), error detection, and efficient data manipulation, acting as a binary addition without carries

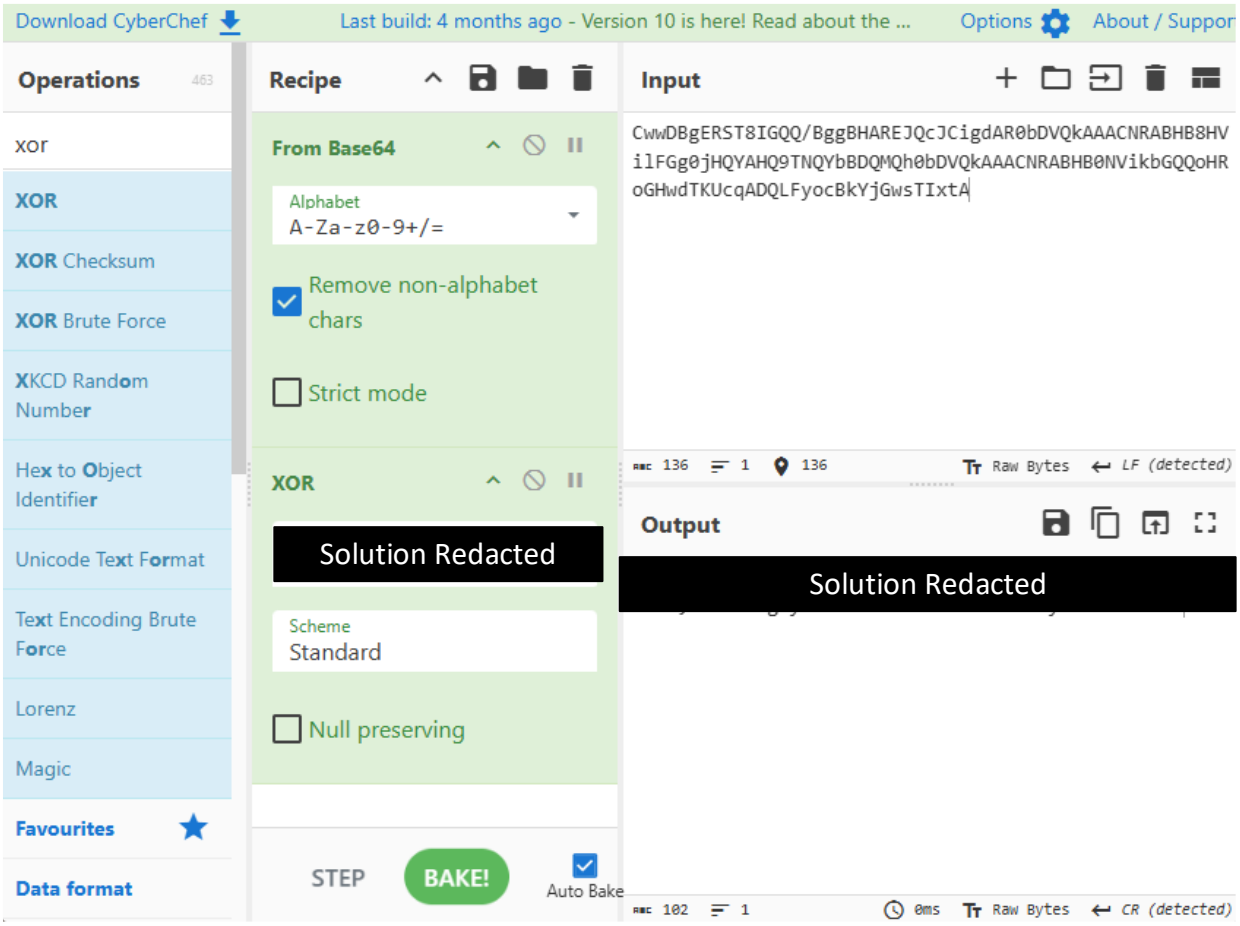
Knowing the founders of derby I knew that I was missing something. But I also knew I was advised to not go too crazy.

And this is when an idea of #BrokenBrilliance, possibly inspired by bourbon hit me. Remembering similar challenges in the past, I was still pretty sure that part of this was an XOR. But what if I had to do something with the Ciphertext light side.

So first I used a ‘From Base64’ on the Ciphertext light side.



Nothing stands out here so now I started XOR-in this output with my phrases listed above in bullet points. CyberChef allows you to stack multiple commands in succession. Feel free to test but ‘Huston, we have a solution!’



For more information on this special challenge please see:

https://trustedsec.com/blog/cracking-the-derbycon-code

Bonus: More Resources

Want to learn more, I suggest these resources:

https://www.cryptool.org/en/cto/

https://manansingh.github.io/Cryptolab-Offline/cryptolab.html

https://www.schneier.com/wp-content/uploads/2016/02/paper-self-study.pdf

https://www.udacity.com/course/applied-cryptography--cs387

https://www.youtube.com/watch?v=ySQl0NhW1J0

https://www.coursera.org/learn/cryptography

http://online.stanford.edu/course/cryptography

https://cryptography101.ca/crypto101-building-blocks/

https://www.coursera.org/learn/crypto

https://www.cybrary.it/course/cryptography/

https://intensecrypto.org/

https://www.khanacademy.org/computing/computer-science/cryptography

http://courses.cs.washington.edu/courses/csep590/06wi/

https://www.youtube.com/watch?v=ZDnShu5V99s

Until Next time!